EXACT BYTE-LEVEL PROBABILITIES FROM TOKENIZED LANGUAGE MODELS FOR FILL-IN-THE-MIDDLE AND CROSS-VOCABULARY ENSEMBLING

Anonymous authors
Paper under double-blind review

ABSTRACT

Tokenized language models (LMs) are trained on token sequences, yet users interact with them through raw text. We show that this mismatch induces *tokenization bias*: conditioning on the canonical tokenization of a prompt implicitly conditions on the event that a token boundary falls at the end of the prompt, which can assign zero probability to the correct continuation even for a perfectly trained model. The failure is acute when prompts end mid-token, as in fill-in-the-middle (FIM) code completion, and the incompatible output spaces of different tokenizers also prevent the direct ensembling of LMs. We formalize valid and *cover* encodings and prove a Byte-Token Representation Lemma that expresses the probability of any byte string exactly in terms of a tokenized LM. Building on a structural result showing that every cover encoding is determined by its final token, we derive an inference-time algorithm that converts any tokenized LM into a statistically equivalent byte-level LM without retraining, requiring one model call per generated byte after a short initialization. On synthetic Markov chains, the method exactly recovers transition probabilities that a tokenized model gets wrong. On the HumanEval random-span FIM benchmark with CodeLlama-7b, byte-level prediction raises greedy pass rates under the suffix-prefix-middle template from 45.0% to 63.9%, outperforming token healing and token alignment. Mapping models with different vocabularies into a shared byte space further enables simple probability averaging, improving over the best member model on MMLU, PIQA, NQ, TriviaQA and HumanEval, while degrading on GSM8K, where members differ widely in capability.

1 INTRODUCTION

Virtually all state-of-the-art language models operate on tokens rather than on raw bytes or characters (Radford et al., 2019; Brown et al., 2020; Touvron et al., 2023; Jiang et al., 2023a). Subword tokenizers such as byte-pair encoding (BPE) (Gage, 1994; Sennrich et al., 2016; Wang et al., 2020), WordPiece (Schuster & Nakajima, 2012) and SentencePiece (Kudo & Richardson, 2018) compress text into shorter sequences, which reduces the quadratic cost of attention (Vaswani et al., 2017) and lets a fixed context window cover more text. However, the deterministic map from bytes to tokens is not innocuous. A tokenized LM learns a distribution over the *canonical* token sequences produced by its tokenizer, whereas the quantity of interest to a user is usually the distribution over the next *characters* given a textual prompt.

These two distributions disagree in a systematic way that we call *tokenization bias*. Consider the code completion example in Figure 1 (left): the prompt ends with `return 5 + a`, and the intended continuation is the variable `aa`. Because the tokenizer would have merged `aa` into a single token had the full word been present, the training data essentially never contains the token `a` followed by the token `a`. Conditioned on the tokenized prompt, the model therefore assigns (near) zero probability to the correct completion. Crucially, this is *not* a modeling error: as we show in Section 3, even a model that perfectly fits the training distribution exhibits the bias, because the tokenized prompt implicitly asserts that a token boundary lies at the end of the prompt. Prompts that

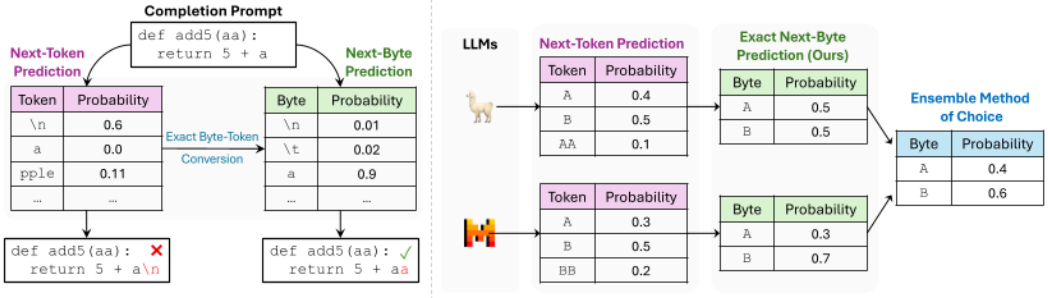


Figure 1: Left: tokenized LMs suffer from tokenization bias when prompts end mid-token, as in this code completion example, where the correct continuation `a` receives zero probability. Our method uses the same model but predicts the correct byte. Right: our method maps the next-token predictions of arbitrary tokenized LMs to statistically equivalent next-byte predictions (Section 3), which enables any ensembling strategy, such as averaging or mixtures of experts.

end mid-token are ubiquitous in fill-in-the-middle (FIM) code completion (Bavarian et al., 2022; Fried et al., 2023; Rozière et al., 2023), where the span to be infilled can start anywhere. Practitioners resort to heuristics such as token healing (Lundberg, 2023; Dagan et al., 2024) and token alignment (Athiwaratkun et al., 2024), which back up one or more prompt tokens and constrain generation, but do not compute the correct conditional distribution.

Tokenization also prevents a second, seemingly unrelated capability: *ensembling* LMs with different vocabularies. The next-token distributions of Llama 2 (Touvron et al., 2023), Mistral (Jiang et al., 2023a) and Yi (Young et al., 2024) live on disjoint supports, so they cannot be averaged directly; existing approaches rely on output reranking (Jiang et al., 2023b), vocabulary alignment (Xu et al., 2024; Huang et al., 2024; Yu et al., 2024) or additional training (Wan et al., 2024).

We address both problems with one tool: an exact, training-free conversion of any tokenized LM into a byte-level LM. Our key observation is that the probability of a byte string can be written as a sum over the probabilities of a small set of token sequences, which we call *cover encodings*, provided that we restrict attention to *valid* encodings, i.e., token sequences the tokenizer could actually produce. Because the byte-level distribution is shared across all tokenizers, models with different vocabularies become directly comparable, and any ensembling strategy can be applied (Figure 1, right).

Contributions. **(i)** We formally characterize tokenization bias as conditioning on a token boundary, and illustrate on Markov chains that it arises even for an ideal model (Section 3.2). **(ii)** We prove a Byte-Token Representation Lemma that maps token-level probabilities to exact byte-level probabilities, and show that each cover encoding is determined by its last token (Section 3.3). This yields an efficient backward search for cover encodings and a byte-level sampling algorithm that needs a single LM call per byte (Section 3.4). **(iii)** On the HumanEval random-span FIM benchmark, byte-level prediction removes most of the degradation caused by mid-token prompts, improving greedy pass rates in SPM mode from 45.0% to 63.9% and outperforming token healing and token alignment; it also improves PSM mode (Section 4.3). **(iv)** We ensemble open-weight LMs with distinct tokenizers by averaging their byte-level predictions, outperforming the best individual model on several knowledge, reasoning and coding benchmarks, and we document when ensembling fails (Section 4.4).

2 Related Work

Tokenization and its side effects. Subword tokenization (Sennrich et al., 2016; Schuster & Nakajima, 2012; Kudo & Richardson, 2018; Wang et al., 2020) is the default for modern LMs; see Mielke et al. (2021) for a historical overview. A growing literature documents its side effects, including degraded multilingual performance (Rust et al., 2021), unfair cost across languages (Petrov et al., 2023), sensitivity of arithmetic to digit tokenization (Singh & Strouse, 2024), and the influence of

tokenizer choice on downstream quality (Ali et al., 2024; Dagan et al., 2024). Theoretical work has analyzed BPE as an optimization problem (Zouhar et al., 2023b), related tokenization quality to information-theoretic efficiency (Zouhar et al., 2023a), studied the role of tokenization when learning Markov sources (Rajaraman et al., 2024), and formalized when token-level models consistently represent distributions over strings (Gastaldi et al., 2024). Phan et al. (2024) study tokenization bias in Markov chains and propose a correction procedure; our work focuses on an efficient exact conversion that applies to arbitrary byte strings, FIM prompts, and cross-vocabulary ensembles of large models.

Marginalizing over tokenizations. Since a string admits many token sequences, several works marginalize over tokenizations when *scoring* text (Cao & Rimell, 2021; Chirkova et al., 2023), typically with importance sampling, and subword regularization (Kudo, 2018; Provilkov et al., 2020) trains models on non-canonical segmentations. Our perspective differs in two respects. First, for deterministic tokenizers, non-canonical (invalid) encodings have zero probability under the data distribution, so we restrict the sum to valid encodings, which makes it exact and tractable. Second, we target *next-byte prediction* and generation, rather than likelihood evaluation of complete strings.

Prompt boundary heuristics. Token healing (Lundberg, 2023; Dagan et al., 2024) removes the last prompt token and constrains the first generated token to start with the removed characters. Token alignment (Athiwaratkun et al., 2024) backs up several tokens and constrains generation to match the removed characters. Both mitigate mid-token prompts, but neither accounts for all valid encodings nor weights them by their probabilities. Our method can be viewed as the principled limit of these heuristics: it marginalizes over every token boundary position that is consistent with the prompt.

Byte-level and token-free models. Character- and byte-level LMs avoid tokenization altogether (Al-Rfou et al., 2019; Xue et al., 2022; Clark et al., 2022; Tay et al., 2022; Yu et al., 2023; Wang et al., 2024), at the cost of longer sequences and the need to train new models from scratch. Our approach is complementary: it obtains byte-level predictions from existing tokenized models at inference time, without retraining.

Ensembling and fusing LMs. Ensembles are a classic way to improve accuracy and calibration (Dietterich, 2000; Lakshminarayanan et al., 2017). For LMs, ensembling is hindered by vocabulary mismatch. Existing approaches select or fuse complete outputs (Jiang et al., 2023b; Wang et al., 2023), align vocabularies through learned or heuristic mappings (Xu et al., 2024; Huang et al., 2024; Yu et al., 2024), optimize mixture weights at test time (Mavromatis et al., 2024), distill multiple models into one (Wan et al., 2024), or transfer a model to a new tokenizer (Minixhofer et al., 2024). Mixture-of-experts architectures (Shazeer et al., 2017; Jiang et al., 2024) share a single vocabulary by construction. In contrast, our byte-level conversion provides an exact common output space, so any combination rule can be applied at every generation step without training or heuristic alignment.

3 METHODOLOGY

3.1 SETUP AND NOTATION

Let $\mathcal{A}$ be the alphabet of bytes and $\mathcal{A}^*$ the set of finite byte strings. We write $x_i^j = (x_i, \ldots, x_j)$ for a substring and $|s|$ for the length of a string s. A tokenizer has a vocabulary $\mathcal{V} \subset \mathcal{A}^*$ of tokens, each of which is a non-empty byte string of length at most L, a deterministic encoder $\mathrm{encode} : \mathcal{A}^* \to \mathcal{V}^*$, and a decoder $\mathrm{decode} : \mathcal{V}^* \to \mathcal{A}^*$ that concatenates the bytes of the tokens. We denote token sequences by $\boldsymbol{t} = (t_1, \ldots, t_k) = t_1^k$ and concatenation by $\oplus$. We consider tokenizers that are lossless, $\mathrm{decode}(\mathrm{encode}(s)) = s$, and focus on BPE (Sennrich et al., 2016) and maximum prefix encoding (MPE, i.e., greedy longest-match as in WordPiece (Schuster & Nakajima, 2012)).

Documents are byte strings drawn from an unknown distribution, and we write $P(x_1^n)$ for the probability that a document begins with x_1^n, so that $P(x_{n+1} \mid x_1^n) = P(x_1^{n+1})/P(x_1^n)$. A tokenized LM is trained on $\mathrm{encode}(\text{document})$ and defines next-token probabilities $P(t_{k+1} \mid t_1^k)$. We write $P(\boldsymbol{t}) = \prod_{i=1}^{k} P(t_i \mid t_1^{i-1})$ for the probability that the token sequence begins with $\boldsymbol{t}$. For the theory, we consider an *ideal* tokenized LM whose distribution equals the distribution of $\mathrm{encode}(\text{document})$; in practice, we apply the same formulas to a trained model.

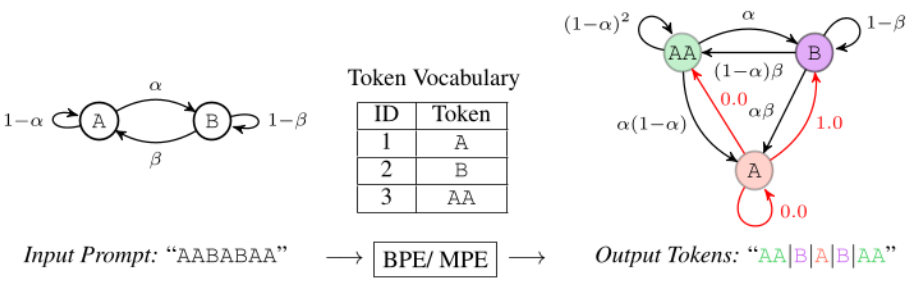


Figure 2: Tokenization bias on a 1st-order Markov chain over $\{\texttt{A}, \texttt{B}\}$ with vocabulary $\{\texttt{A}, \texttt{B}, \texttt{AA}\}$. Given the context token "A", the tokenized model never samples "A" as the next byte, whereas the true probability is $1 - \alpha$. In practice, the same bias occurs when prompts end with tokens that are prefixes of other tokens, which is common in FIM tasks.

3.2 Tokenization Bias

The standard way to obtain next-character predictions from a tokenized LM is to tokenize the prompt and read off the distribution of the first byte of the next token:

$$\hat{P}(x_{n+1} = a \mid x_1^n) \;=\; \sum_{\tau \in \mathcal{V}:\, \tau_1 = a} P\big(\tau \mid \mathrm{encode}(x_1^n)\big). \tag{1}$$

Let B_n denote the event that the canonical tokenization of the document has a token boundary right after position n. For the tokenizers we consider (see Proposition 1 below), a document that begins with x_1^n and has a boundary at n is tokenized as $\mathrm{encode}(x_1^n)$ followed by further tokens. Hence, even for an ideal model, Equation 1 computes

$$\hat{P}(x_{n+1} = a \mid x_1^n) \;=\; P\big(x_{n+1} = a \mid x_1^n,\, B_n\big) \;\neq\; P\big(x_{n+1} = a \mid x_1^n\big) \quad \text{in general.} \tag{2}$$

We call the discrepancy in Equation 2 *tokenization bias*. It arises whenever knowing that a token ends at position n is informative about the next byte, which is precisely the case when the prompt ends with a token that is a prefix of other tokens.

Example. Figure 2 shows a 1st-order Markov chain over $\{\texttt{A}, \texttt{B}\}$ that stays at A with probability $1-\alpha$ and at B with probability $1 - \beta$. With vocabulary $\{\texttt{A}, \texttt{B}, \texttt{AA}\}$, both BPE and MPE merge every pair of consecutive As, so the string AABABAA is tokenized as AA | B | A | B | AA. A standalone token A can therefore only be followed by B: the induced token process assigns $P(\texttt{A} \mid \texttt{A}) = P(\texttt{AA} \mid \texttt{A}) = 0$ and $P(\texttt{B} \mid \texttt{A}) = 1$. A prompt ending in BA is encoded with a final token A, and Equation 1 yields $\hat{P}(\texttt{A} \mid \ldots \texttt{BA}) = 0$, although the true probability is $1 - \alpha$. No amount of training removes this error, because it is a property of the token process itself.

3.3 Valid Encodings, Cover Encodings, and the Byte-Token Representation Lemma

Our approach replaces conditioning on one tokenization with an exact marginalization. The key is to know which token sequences can occur at all.

Definition 1 (Valid encoding)**.** *A token sequence $\boldsymbol{t} \in \mathcal{V}^*$ is* valid *if* $\mathrm{encode}(\mathrm{decode}(\boldsymbol{t})) = \boldsymbol{t}$*, and* invalid *otherwise.*

Under the ideal model, invalid sequences are never generated, $P(\boldsymbol{t}) = 0$, because they never appear in tokenized training data. In Figure 2, A | A is invalid since $\mathrm{encode}(\texttt{AA}) = \texttt{AA}$. We rely on the following property, proven in Appendix A.

Proposition 1 (Prefix closure)**.** *For BPE and MPE, every prefix of a valid encoding is valid: if t_1^k is valid, then t_1^j is valid and equals* $\mathrm{encode}(\mathrm{decode}(t_1^j))$ *for all $j \leq k$.*

Definition 2 (Cover encodings)**.** *The* cover encodings *of a byte string x_1^n are the valid token sequences $\boldsymbol{t} = t_1^k$ such that x_1^n is a prefix of* $\mathrm{decode}(\boldsymbol{t})$ *and* $\mathrm{decode}(t_1^{k-1})$ *is a strict prefix of x_1^n. We denote this set by* $\mathrm{cover}(x_1^n)$.

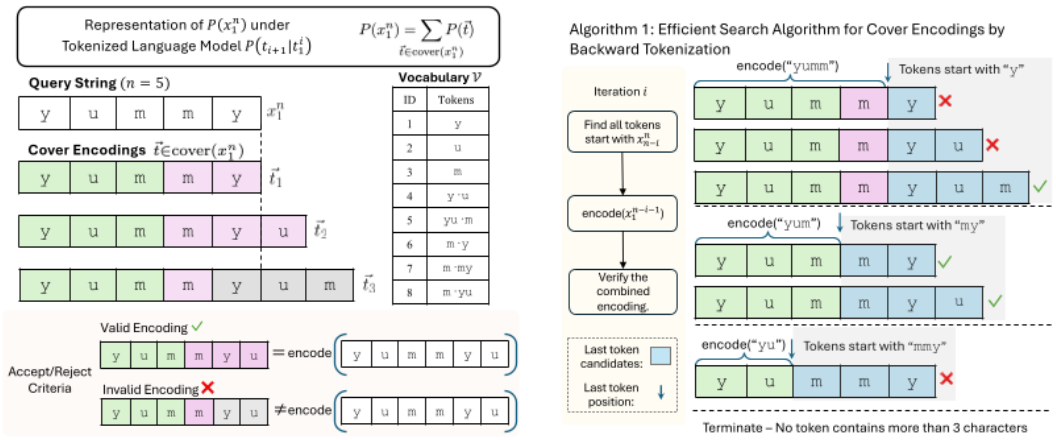


Figure 3: Left: representation of $P(x_1^n)$ with a tokenized LM, with examples of cover encodings and of valid and invalid encodings. Right: illustration of Algorithm 1 on the query string `yummy` with a BPE vocabulary. Green ticks and red crosses denote valid and invalid encodings (Definition 1). The search terminates once the suffix exceeds the maximum token length L.

In words, a cover encoding tokenizes the string up to some position $j < n$ and then ends with a single token that covers the remaining bytes x_{j+1}^n and may extend beyond position n (Figure 3, left). The canonical encoding $\text{encode}(x_1^n)$ is always a cover encoding.

Lemma 1 (Byte-Token Representation)**.** *Under the ideal tokenized LM, for every byte string x_1^n,*

$$P(x_1^n) = \sum_{\boldsymbol{t} \in \text{cover}(x_1^n)} P(\boldsymbol{t}). \tag{3}$$

Proof sketch. Let $T = \text{encode}(\text{document})$. If the document begins with x_1^n, let k^* be the smallest index such that $|\,\text{decode}(T_1^{k^*})| \geq n$. By Proposition 1, $T_1^{k^*}$ is valid, and by construction it belongs to $\text{cover}(x_1^n)$. Conversely, if T begins with some $\boldsymbol{t} \in \text{cover}(x_1^n)$, the document begins with x_1^n. Finally, no two distinct cover encodings can both be prefixes of T, since the shorter one would already reach length n, contradicting the definition for the longer one. Hence the event that the document begins with x_1^n is the disjoint union of the events that T begins with $\boldsymbol{t}$, $\boldsymbol{t} \in \text{cover}(x_1^n)$. A full proof is in Appendix A. □

Lemma 1 turns any tokenized LM into a byte-level LM: next-byte probabilities follow from $P(x_{n+1} \mid x_1^n) = P(x_1^{n+1})/P(x_1^n)$, and each $P(\boldsymbol{t})$ is computed by the chain rule with the LM. It remains to enumerate the cover encodings efficiently.

Structure of cover encodings. Naively, the number of token sequences that decode to a string grows exponentially with its length. Validity collapses this set dramatically:

Proposition 2 (Cover encodings are determined by their last token)**.** *Every $\boldsymbol{t} \in \text{cover}(x_1^n)$ has the form $\boldsymbol{t} = \text{encode}(x_1^j) \oplus \tau$, where $n - L \leq j < n$ and $\tau \in \mathcal{V}$ has x_{j+1}^n as a prefix. Consequently, $|\,\text{cover}(x_1^n)| \leq \sum_{i=0}^{L-1} \big|\{\tau \in \mathcal{V} :\ x_{n-i}^n \text{ is a prefix of } \tau\}\big|$.*

Proof. Write $\boldsymbol{t} = t_1^k$ and $\text{decode}(t_1^{k-1}) = x_1^j$ with $j < n$. By Proposition 1, t_1^{k-1} is valid, so $t_1^{k-1} = \text{encode}(x_1^j)$. The last token t_k must contain x_{j+1}^n as a prefix, so $n - j \leq |t_k| \leq L$. □

Proposition 2 implies that we never need to search over the full history: it suffices to look backwards at most L bytes, and for each candidate boundary j the prefix is fixed to the canonical encoding $\text{encode}(x_1^j)$. This gives Algorithm 1, illustrated in Figure 3 (right). The probability of each cover encoding factorizes as $P(\text{encode}(x_1^j) \oplus \tau) = P(\text{encode}(x_1^j))\, P(\tau \mid \text{encode}(x_1^j))$, so all of them can be scored with at most L forward passes, one per candidate boundary, which can be batched and share a key-value cache whenever the canonical prefixes coincide.

Algorithm 1 Cover encoding search by backward tokenization

Require: byte string x_1^n, vocabulary $\mathcal{V}$ with maximum token length L, tokenizer encode, tokenized LM P

$\mathcal{C} \leftarrow \emptyset$
for $i = 0, 1, \ldots, \min(n, L) - 1$ **do** ▷ the last token starts at position $n - i$
  $\boldsymbol{p} \leftarrow \mathrm{encode}(x_1^{n-i-1})$ ▷ canonical encoding of the remaining prefix
  for each $\tau \in \mathcal{V}$ such that x_{n-i}^n is a prefix of τ **do**
    if $\mathrm{encode}(\mathrm{decode}(\boldsymbol{p} \oplus \tau)) = \boldsymbol{p} \oplus \tau$ **then** ▷ validity check (Definition 1)
      $\mathcal{C} \leftarrow \mathcal{C} \cup \{\boldsymbol{p} \oplus \tau\}$ with score $P(\boldsymbol{p})\, P(\tau \mid \boldsymbol{p})$
    end if
  end for
end for
return $\mathcal{C} = \mathrm{cover}(x_1^n)$ and the scores $\{P(\boldsymbol{t})\}_{\boldsymbol{t} \in \mathcal{C}}$

3.4 Exact next-byte prediction and sampling

Algorithm 1 is only needed once, for the prompt. During generation, the cover encodings of x_1^{n+1} can be obtained from those of x_1^n with a single LM call. Partition $\mathrm{cover}(x_1^n)$ into the encodings that extend strictly beyond position n and the unique encoding that ends exactly at n, which by validity is $\boldsymbol{t}^\star = \mathrm{encode}(x_1^n)$. For each byte $a \in \mathcal{A}$, define

$$\mathcal{C}_{n+1}(a) = \big\{\boldsymbol{t} \in \mathrm{cover}(x_1^n) : \ |\mathrm{decode}(\boldsymbol{t})| > n, \ \mathrm{decode}(\boldsymbol{t})_{n+1} = a\big\}, \tag{4}$$

$$\bar{\mathcal{C}}_{n+1}(a) = \big\{\boldsymbol{t}^\star \oplus \tau : \ \tau \in \mathcal{V}, \ \tau_1 = a, \ \boldsymbol{t}^\star \oplus \tau \text{ is valid}\big\}. \tag{5}$$

Proposition 3 (Recursive cover update)**.** *For every $a \in \mathcal{A}$, $\mathrm{cover}(x_1^n a) = \mathcal{C}_{n+1}(a) \cup \bar{\mathcal{C}}_{n+1}(a)$, the union being disjoint. Therefore, under the ideal model,*

$$P(x_{n+1} = a \mid x_1^n) \ = \ \frac{\sum_{\boldsymbol{t} \in \mathcal{C}_{n+1}(a)} P(\boldsymbol{t}) \ + \ P(\boldsymbol{t}^\star) \sum_{\boldsymbol{t}^\star \oplus \tau \in \bar{\mathcal{C}}_{n+1}(a)} P(\tau \mid \boldsymbol{t}^\star)}{\sum_{b \in \mathcal{A}} \Big[\sum_{\boldsymbol{t} \in \mathcal{C}_{n+1}(b)} P(\boldsymbol{t}) \ + \ P(\boldsymbol{t}^\star) \sum_{\boldsymbol{t}^\star \oplus \tau \in \bar{\mathcal{C}}_{n+1}(b)} P(\tau \mid \boldsymbol{t}^\star) \Big]}. \tag{6}$$

The first term in the numerator reuses probabilities that were already computed for $\mathrm{cover}(x_1^n)$; only the second term requires a new forward pass, on $\boldsymbol{t}^\star$. Under the ideal model, the denominator equals $P(x_1^n)$ by Lemma 1; for a trained model, which places a small amount of mass on invalid continuations, normalizing over b discards that mass consistently. Comparing Equation 6 with Equation 1 makes the source of tokenization bias explicit: the naive estimate keeps only the second term and ignores all encodings in which a token straddles position n.

The full procedure (Algorithm 2 in Appendix B, illustrated in Figure 4) proceeds in five steps: (1) obtain $\mathrm{cover}(x_1^n)$, from Algorithm 1 at the first step or from the previous iteration otherwise; (2) regroup the encodings that extend beyond n by their $(n{+}1)$-th byte (Equation 4); (3) run the LM once on $\boldsymbol{t}^\star = \mathrm{encode}(x_1^n)$, accept or reject each next token through the validity check, and group the accepted ones by their first byte (Equation 5); (4) combine both groups into $\mathrm{cover}(x_1^n a)$ for every byte a and compute Equation 6; (5) sample (or select greedily) x_{n+1} and keep $\mathrm{cover}(x_1^{n+1})$ for the next step. Note that $\mathrm{encode}(x_1^{n+1})$ is itself a member of $\mathrm{cover}(x_1^{n+1})$, so its probability is available at the next iteration. All computations are carried out in log space.

Cost. After initialization (at most L batched forward passes), the method requires one LM call per generated *byte*, compared with one call per *token* for standard decoding; the overhead thus scales with the average number of bytes per token. The validity checks only require the tokenizer, not the LM, and only tokens with non-negligible probability affect Equation 6.

3.5 Applications

Fill-in-the-middle. FIM models are prompted with sentinel tokens, either in prefix-suffix-middle (PSM) order, <PRE> prefix <SUF> suffix <MID>, or in suffix-prefix-middle (SPM) order, in which the prompt ends with the prefix and the middle continues it directly (Bavarian et al., 2022; Rozière et al., 2023). In SPM, the prompt therefore ends mid-token whenever the span to be infilled starts

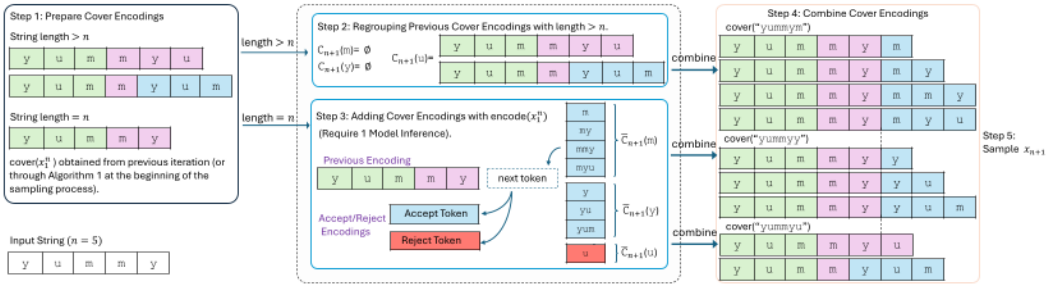


Figure 4: Illustration of sampling x_{n+1} from a tokenized LM $P(t_{k+1} \mid t_1^k)$, continuing the example of Figure 3. Cover encodings longer than n are regrouped by their next byte (step 2), and the canonical encoding is extended by one LM call with accept/reject validity checks (step 3); the two groups are combined (step 4) before sampling (step 5). Probabilities $P(\boldsymbol{t})$ are omitted for simplicity.

inside a word. We treat sentinel tokens as atomic boundaries, so the cover search never crosses them, and apply Algorithm 1 to the text that follows the last sentinel. End-of-sequence tokens are treated as an additional symbol of the byte alphabet, so generation stops when this symbol is selected.

Byte-level greedy and beam search. Equation 6 defines a proper autoregressive distribution over bytes, so standard decoding strategies apply. Greedy decoding selects $\arg\max_a P(x_{n+1}{=}a \mid x_1^n)$, which aggregates the mass of *all* tokens starting with byte a and is therefore a different decision rule from token-level greedy decoding. For beam search with width B, each hypothesis is a byte string with its own cover set, scored by $\log P(x_1^n)$.

Cross-vocabulary ensembling. Because Equation 6 lives in the common space $\mathcal{A}$, predictions of models $P_1, \ldots, P_M$ with different tokenizers can be combined directly. We use the simplest rule, the average $P_{\text{ens}}(x_{n+1} \mid x_1^n) = \frac{1}{M}\sum_{m=1}^{M} P_m(x_{n+1} \mid x_1^n)$. After a byte is selected, every member updates its own cover set with Proposition 3. Other rules, such as weighted averaging or mixtures of experts, can be plugged in without modification.

4 Experiments

We evaluate (i) whether the byte-token conversion recovers the true byte-level distribution in a controlled setting, (ii) whether it fixes the mid-token failure mode in FIM code completion, and (iii) whether the byte space enables effective ensembling of models with different vocabularies.

4.1 Setup

Models. For the synthetic study, we use 1st- and 3rd-order Markov chains over $\{\texttt{A}, \texttt{B}\}$ and a GPT-2 architecture (Radford et al., 2019) with 6 hidden layers trained on the tokenized output of the 3rd-order chain. For FIM, we use CodeLlama-7b (Rozière et al., 2023). For general-purpose evaluation, we use Llama2-7B (Touvron et al., 2023), Yi-1.5-6B (Young et al., 2024) and Mistral-7B-v0.3 (Jiang et al., 2023a), and for code generation CodeLlama2-7b, CodeLlama2-13b (Rozière et al., 2023) and Yi-Coder-1.5B. All are Transformer-based LMs and are used without any weight updates.

Tasks and metrics. For FIM, we use the HumanEval random-span infilling benchmark (Bavarian et al., 2022) in both PSM and SPM modes, reporting greedy pass rates and pass@10, where pass@10 is the maximum over a temperature sweep in $[0, 1]$. For knowledge and reasoning, we use MMLU (Hendrycks et al., 2021), PIQA (Bisk et al., 2020), Natural Questions (NQ) (Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017) and GSM8K (Cobbe et al., 2021), reporting accuracy. For code generation, we use HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021), reporting pass@1. For each model, we compare predictions from the standard token-level distribution (*Token*) with predictions from our byte-level distribution (*Byte*) obtained from the same model.

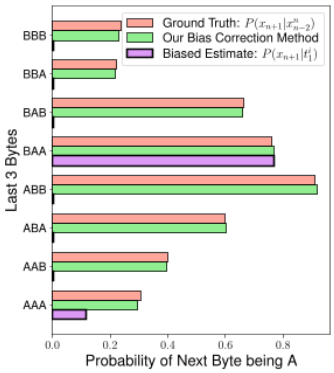


Figure 5: Tokenization bias on a 3rd-order Markov chain. Our byte-token conversion accurately recovers $P(x_{n+1} \mid x_{n-2}^n)$ of the chain.

Table 1: Code generation (pass@1, %). The ensemble averages the byte-level predictions of the top-2 models (Yi-Coder-1.5B and CodeLlama2-13b).

	HumanEval		MBPP	
Model	Token	Byte	Token	Byte
CodeLlama2-7b	32.3	28.7	40.6	42.4
CodeLlama2-13b	35.8	33.5	47.6	47.4
Yi-Coder-1.5B	38.4	36.5	52.8	**53.6**
Top-2 ensemble (ours)	–	**42.1**	–	**53.6**

Table 2: Beam width ablation for byte-level decoding on HumanEval random-span FIM with CodeLlama-7b (pass rate, %).

Beam width	1	2	4
PSM	65.7	66.8	**67.7**
SPM	63.9	65.9	**66.8**

	PSM		SPM	
Predictions	Greedy	@10	Greedy	@10
Token-level	64.9	84.0	45.0	66.5
+ healing	|	|	62.7	83.8
+ alignment	|	|	63.0	82.9
Byte-level	**65.7**	**84.1**	**63.9**	**84.3**

@10 here is the max over the temperature sweep on the right

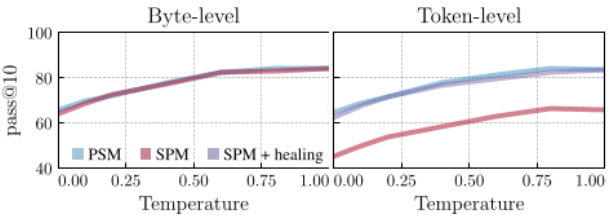


Figure 6: Pass rates on the HumanEval random-span benchmark (Bavarian et al., 2022) with CodeLlama-7b for infilling Python. Left: greedy pass rates and pass@10 (maximum over the temperature sweep on the right) for the PSM and SPM templates; token healing (Dagan et al., 2024) and alignment (Athiwaratkun et al., 2024) are applied in SPM mode. Right: pass@10 as a function of temperature. The SPM template forces predictions from the middle of a token, which byte-level prediction handles successfully.

Baselines. Besides standard token-level prediction, we compare against token healing (Lundberg, 2023; Dagan et al., 2024) and token alignment (Athiwaratkun et al., 2024) for FIM, both applied in SPM mode where they are relevant. For ensembling, we compare against the individual member models and, for the multiple-choice tasks MMLU and PIQA, against top-2 majority voting. All ensembles combine the top-2 individual models per benchmark by averaging their byte-level probabilities.

4.2 Tokenization bias on synthetic data

In the 1st-order chain of Figure 2, the token-level model assigns probability 0.0 to `A` following a context that ends in the token `A`, whereas the ground-truth probability is $1 - \alpha$, confirming the analysis of Section 3.2. Figure 5 reports the 3rd-order chain, for which we trained the GPT-2 model on tokenized sequences. For each of the eight 3-byte contexts, we compare the ground-truth probability that the next byte is `A`, the naive estimate $P(x_{n+1} \mid t_1^k)$ of Equation 1, and our byte-level estimate. The naive estimate is severely biased: for most contexts it places (close to) zero probability on `A` even though the ground truth is substantial (e.g., above 0.6 for `BAB`, `ABB` and `ABA`). In contrast, our bias-corrected estimate, computed from the very same trained model, closely matches the ground truth for all contexts. This confirms both that tokenization bias is severe for trained models and that Lemma 1 recovers the original transition probabilities without retraining.

Table 3: Knowledge and reasoning benchmarks (accuracy, %). For each model, Token and Byte denote predictions from the token-level distribution and from our byte-level conversion of the same model. Voting and the ensemble combine the top-2 individual models per benchmark; voting is only applicable to the multiple-choice tasks. Best result per benchmark in bold.

	MMLU		PIQA		NQ		TriviaQA		GSM8K	
Model	Token	Byte	Token	Byte	Token	Byte	Token	Byte	Token	Byte
Llama2-7B	45.4	45.7	78.1	78.2	25.0	23.4	58.4	58.1	15.1	12.5
Yi-1.5-6B	63.4	63.4	78.5	78.5	22.8	22.7	53.7	53.4	61.3	**61.5**
Mistral-7B-v0.3	62.1	62.1	80.1	80.3	28.5	28.8	63.6	63.5	39.0	39.0
Voting (top-2)	–	62.1	–	80.1	–	–	–	–	–	–
Top-2 ensemble (ours)	–	**65.4**	–	**80.7**	–	**30.0**	–	**64.2**	–	55.8

4.3 Fill-in-the-middle code completion

Figure 6 summarizes the FIM results. **Mid-token prompts break token-level prediction.** Switching from PSM to SPM reduces the token-level greedy pass rate from 64.9% to 45.0% and pass@10 from 84.0% to 66.5%. The right panel shows that the gap persists across all sampling temperatures, so it cannot be fixed by sampling more diversely; the issue is that the correct continuation is out of distribution given the canonical prompt tokenization. **Byte-level prediction removes the degradation.** With byte-level predictions from the same model, the SPM greedy pass rate rises to 63.9% (+18.9 points) and pass@10 to 84.3% (+17.8 points), on par with PSM; the byte-level pass@10 curves for PSM and SPM nearly coincide across temperatures. **Comparison with heuristics.** Token healing (62.7% greedy, 83.8% pass@10) and token alignment (63.0%, 82.9%) recover most of the gap, confirming that the SPM failure stems from prompt boundaries, but both remain below byte-level prediction in SPM mode (63.9%, 84.3%). **Gains also in PSM.** Even in PSM mode, where prompts end with a sentinel token and tokenization issues should be minimal, byte-level greedy decoding improves over token-level greedy decoding (65.7% vs. 64.9%), while pass@10 is essentially unchanged (84.1% vs. 84.0%). A possible explanation is that byte-level greedy decoding aggregates the probability of all tokens sharing a first byte (Section 3.5), and that the middle span itself may start inside a word of the prefix; we did not isolate these factors.

Error analysis. Inspecting failures of the token-level model in SPM mode, we find cases in which the prompt ends in the middle of an identifier, for example in variables such as `delimeter`, `collatz` and `palindrome`. The token-level model fails to complete the word because the remaining characters do not form a token that can follow the canonical prompt tokenization, while the byte-level method identifies the correct completion.

Ablation: byte-level beam search. Table 2 varies the beam width of byte-level decoding on the FIM task. Wider beams yield consistent gains in both modes: from 65.7% to 67.7% in PSM (+2.0 points) and from 63.9% to 66.8% in SPM (+2.9 points) when going from width 1 (greedy) to width 4. Hence the byte-level distribution is compatible with standard search-based decoding, and its benefits are complementary to those of beam search.

4.4 Byte-level conversion and cross-vocabulary ensembling

Tables 3 and 1 report token-level and byte-level results for individual models, and the results of ensembling the top-2 models of each benchmark in byte space.

Individual models. On benchmarks whose prompts end at natural boundaries, there is no mid-token issue to fix, and byte-level and token-level results are close on most tasks: MMLU, PIQA and TriviaQA differ by at most 0.3 points for every model, and Mistral-7B-v0.3 is within 0.3 points on all five benchmarks. There are, however, notable exceptions in both directions. Byte-level prediction reduces accuracy for Llama2-7B on NQ (-1.6) and GSM8K (-2.6), and reduces HumanEval pass@1 for all three code models (-3.6, -2.3 and -1.9 points), whereas it improves MBPP for CodeLlama2-7b ($+1.8$) and Yi-Coder-1.5B ($+0.8$). For an ideal model, byte-level and token-level *sampling* define the same distribution over strings; the differences therefore stem from decoding

rules and model imperfections. In particular, greedy decoding over bytes is not equivalent to greedy decoding over tokens, and the byte-level conversion discards the mass that a trained model places on invalid encodings. We have not disentangled these effects, and the HumanEval regressions indicate that byte-level greedy decoding is not uniformly beneficial for open-ended generation.

Ensembles across vocabularies. The byte space allows us to average models whose tokenizers are incompatible. On MMLU, the ensemble of Yi-1.5-6B and Mistral-7B-v0.3 reaches 65.4%, improving by 2.0 points over the best member (63.4%) and by 3.3 points over top-2 voting (62.1%). The ensemble also outperforms the best individual result on PIQA (80.7% vs. 80.3%), NQ (30.0% vs. 28.8%) and TriviaQA (64.2% vs. 63.6%). For code, the ensemble of Yi-Coder-1.5B and CodeLlama2-13b achieves 42.1% on HumanEval, 3.7 points above the best individual model (38.4%), even though each member's byte-level score is lower than its token-level score; on MBPP it matches the best member (53.6%).

Limitation of ensembling. Ensembling does not help when members differ widely in capability. On GSM8K, Yi-1.5-6B outperforms Mistral-7B-v0.3 by more than 20 points (61.5% vs. 39.0% in byte space), and the uniform average (55.8%) falls 5.7 points below the stronger model. In multi-step generation such as math reasoning, a single poor byte choice induced by the weaker model can derail the solution, so uniform averaging is ill-suited to unequal members; weighted or input-dependent combination rules are natural remedies that our framework supports but that we have not evaluated.

Statistical caveats. All numbers are single evaluations on the standard test sets; we do not report confidence intervals. Several differences, especially those below one point (e.g., PIQA, TriviaQA, and the PSM pass@10), are small relative to the sampling variability expected on benchmarks of this size and should be interpreted with caution. The large effect in SPM mode (+18.9 greedy points) is unlikely to be explained by noise, whereas the moderate ensemble gains on MMLU (+2.0) and HumanEval (+3.7, i.e., about six of 164 problems) would benefit from confirmation with significance tests.

5 Conclusion

We identified tokenization bias as an intrinsic property of tokenized LMs: conditioning on the canonical tokenization of a prompt implicitly conditions on a token boundary at its end, which can make the correct continuation impossible. By restricting attention to valid encodings, we derived the Byte-Token Representation Lemma, showed that cover encodings are determined by their final token, and obtained an exact, training-free algorithm that turns any tokenized LM into a statistically equivalent byte-level LM with one model call per byte. Empirically, the conversion recovers ground-truth transition probabilities on Markov chains, removes most of the degradation caused by mid-token FIM prompts (45.0%→63.9% greedy pass rate in SPM mode), outperforms token healing and token alignment, and enables simple averaging of models with different vocabularies, which improves over the best member on several benchmarks.

Limitations and future work. First, byte-level decoding requires one forward pass per byte rather than per token, which increases latency roughly by the average number of bytes per token; speculative or multi-byte decoding could reduce this cost. Second, our guarantees hold for an ideal model and for tokenizers satisfying prefix closure; production tokenizers with pre-tokenization rules or normalization require care, and trained models place non-zero mass on invalid encodings. Third, byte-level greedy decoding does not uniformly improve individual models, as the HumanEval results show, and uniform ensembling hurts when members are unequal, as on GSM8K. Finally, our evaluation reports single runs without confidence intervals. Promising directions include learned or adaptive ensemble weights, byte-level mixtures of experts over specialized open-weight models, and applying the conversion to other settings where prompt boundaries matter, such as constrained decoding.

Reproducibility Statement

All quantities used by our method are defined in Section 3: Algorithm 1 specifies the cover search, Equation 6 and Algorithm 2 (Appendix B) specify next-byte prediction, and Appendix A contains

the proofs. The method requires no training and only uses publicly available model weights, tokenizers and benchmarks.

REFERENCES

Rami Al-Rfou, Dokook Choe, Noah Constant, Mandy Guo, and Llion Jones. Character-level language modeling with deeper self-attention. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pp. 3159–3166, 2019.

Mehdi Ali, Michael Fromm, Klaudia Thellmann, Richard Rutmann, Max Lübbering, Johannes Leveling, Katrin Klug, Jan Ebert, Niclas Doll, Jasper Schulze Buschhoff, et al. Tokenizer choice for LLM training: Negligible or crucial? In *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024.

Ben Athiwaratkun, Shiqi Wang, Mingyue Shang, Yuchen Tian, Zijian Wang, Sujan Kumar Gonugondla, Sanjay Krishna Gouda, Rob Kwiatkowski, Ramesh Nallapati, and Bing Xiang. Token alignment via character matching for subword completion. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255*, 2022.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 7432–7439, 2020.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Kris Cao and Laura Rimell. You should evaluate your language model on marginal likelihood over tokenisations. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 2104–2114, 2021.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Nadezhda Chirkova, Germán Kruszewski, Jos Rozen, and Marc Dymetman. Should you marginalize over possible tokenizations? In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 1–12, 2023.

Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. CANINE: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91, 2022.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Gautier Dagan, Gabriel Synnaeve, and Baptiste Rozière. Getting the most out of your tokenizer for pre-training and domain adaptation. In *International Conference on Machine Learning (ICML)*, 2024.

Thomas G. Dietterich. Ensemble methods in machine learning. In *International Workshop on Multiple Classifier Systems*, pp. 1–15, 2000.

Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. InCoder: A generative model for code infilling and synthesis. In *International Conference on Learning Representations (ICLR)*, 2023.

Philip Gage. A new algorithm for data compression. *The C Users Journal*, 12(2):23–38, 1994.

Juan Luis Gastaldi, John Terilla, Luca Malagutti, Brian DuSell, Tim Vieira, and Ryan Cotterell. The foundations of tokenization: Statistical and computational concerns. *arXiv preprint arXiv:2407.11606*, 2024.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.

Yichong Huang, Xiaocheng Feng, Baohang Li, Yang Xiang, Hui Wang, Bing Qin, and Ting Liu. Ensemble learning for heterogeneous large language models with deep parallel collaboration. *arXiv preprint arXiv:2404.12715*, 2024.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023a.

Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. LLM-Blender: Ensembling large language models with pairwise ranking and generative fusion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 14165–14178, 2023b.

Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1601–1611, 2017.

Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 66–75, 2018.

Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 66–71, 2018.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.

Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Scott Lundberg. The art of prompt design: Prompt boundaries and token healing. Towards Data Science (blog post), 2023.

Costas Mavromatis, Petros Karypis, and George Karypis. Pack of LLMs: Model fusion at test-time via perplexity optimization. *arXiv preprint arXiv:2404.11531*, 2024.

Sabrina J. Mielke, Zaid Alyafeai, Elizabeth Salesky, Colin Raffel, Manan Dey, Matthias Gallé, Arun Raja, Chenglei Si, Wilson Y. Lee, Benoît Sagot, and Samson Tan. Between words and characters: A brief history of open-vocabulary modeling and tokenization in NLP. *arXiv preprint arXiv:2112.10508*, 2021.

Benjamin Minixhofer, Edoardo Maria Ponti, and Ivan Vulić. Zero-shot tokenizer transfer. *arXiv preprint arXiv:2405.07883*, 2024.

Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Buu Phan, Marton Havasi, Matthew Muckley, and Karen Ullrich. Understanding and mitigating tokenization bias in language models. *arXiv preprint arXiv:2406.16829*, 2024.

Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. BPE-dropout: Simple and effective subword regularization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1882–1892, 2020.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI blog*, 2019.

Nived Rajaraman, Jiantao Jiao, and Kannan Ramchandran. Toward a theory of tokenization in LLMs. *arXiv preprint arXiv:2404.08335*, 2024.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. How good is your tokenizer? on the monolingual performance of multilingual language models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 3118–3135, 2021.

Mike Schuster and Kaisuke Nakajima. Japanese and Korean voice search. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5149–5152, 2012.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1715–1725, 2016.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.

Aaditya K. Singh and D. J. Strouse. Tokenization counts: The impact of tokenization on arithmetic in frontier LLMs. *arXiv preprint arXiv:2402.14903*, 2024.

Yi Tay, Vinh Q. Tran, Sebastian Ruder, Jai Gupta, Hyung Won Chung, Dara Bahri, Zhen Qin, Simon Baumgartner, Cong Yu, and Donald Metzler. Charformer: Fast character transformers via gradient-based subword tokenization. In *International Conference on Learning Representations (ICLR)*, 2022.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Fanqi Wan, Xinting Huang, Deng Cai, Xiaojun Quan, Wei Bi, and Shuming Shi. Knowledge fusion of large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

Changhan Wang, Kyunghyun Cho, and Jiatao Gu. Neural machine translation with byte-level subwords. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 9154–9160, 2020.

Junxiong Wang, Tushaar Gangavarapu, Jing Nathan Yan, and Alexander M. Rush. MambaByte: Token-free selective state space model. *arXiv preprint arXiv:2401.13660*, 2024.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Yangyifan Xu, Jinliang Lu, and Jiajun Zhang. Bridging the gap between different vocabularies for LLM ensemble. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pp. 7140–7152, 2024.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022.

Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Heng Li, Jiangcheng Zhu, Jianqun Chen, Jing Chang, et al. Yi: Open foundation models by 01.AI. *arXiv preprint arXiv:2403.04652*, 2024.

Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. MEGABYTE: Predicting million-byte sequences with multiscale transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Yao-Ching Yu, Chun-Chih Kuo, Ziqi Ye, Yu-Cheng Chang, and Yueh-Se Li. Breaking the ceiling of the LLM community by treating token generation as a classification for ensembling. *arXiv preprint arXiv:2406.12585*, 2024.

Vilém Zouhar, Clara Meister, Juan Luis Gastaldi, Li Du, Mrinmaya Sachan, and Ryan Cotterell. Tokenization and the noiseless channel. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 5184–5207, 2023a.

Vilém Zouhar, Clara Meister, Juan Luis Gastaldi, Li Du, Tim Vieira, Mrinmaya Sachan, and Ryan Cotterell. A formal perspective on byte-pair encoding. In *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 598–614, 2023b.

A PROOFS

Throughout, documents are finite byte strings, $T = \mathrm{encode}(\text{document})$ is the canonical token sequence of a random document, and $P(\boldsymbol{t})$ denotes the probability that T begins with $\boldsymbol{t}$, which is what an ideal tokenized LM computes by the chain rule.

Proof of Proposition 1 (prefix closure). Let $\boldsymbol{t} = t_1^k = \mathrm{encode}(s)$ and fix $j \le k$; write $u = \mathrm{decode}(t_1^j)$, so that $s = uv$ for some v. We show $\mathrm{encode}(u) = t_1^j$.

MPE. MPE repeatedly emits the longest token that is a prefix of the remaining string. We argue by induction over the emitted tokens. Suppose the first $i < j$ tokens of $\mathrm{encode}(u)$ equal t_1^i, and let r be the remaining part of u, so that rv' is the remaining part of s for some v'. On s, MPE emitted t_{i+1}, the longest token that is a prefix of rv'; since t_{i+1} is also a prefix of r (because $i+1 \le j$), and any token that is a prefix of r is also a prefix of rv' and hence no longer than t_{i+1}, MPE emits t_{i+1} on u as well. After j steps u is exhausted, so $\mathrm{encode}(u) = t_1^j$.

BPE. BPE starts from single bytes and repeatedly merges the adjacent pair with the lowest merge rank, breaking ties from left to right, until no mergeable pair remains. Since merges only coarsen the segmentation and the final segmentation of s has a boundary between u and v, no merge in the trajectory on s crosses that boundary. We claim that running BPE on u performs exactly the subsequence of merges of the trajectory on s that lie within u. Consider any state in which the parts within u coincide. In the trajectory on s, merges within v do not change the part within u; when the next merge within u occurs, it is the lowest-ranked pair among all pairs of s, hence also among the pairs of u, and in case of ties it is the leftmost one, since all pairs of u lie to the left of pairs involving v. This is exactly the merge BPE selects on u. Finally, when BPE terminates on s, no mergeable pair remains in u, so BPE on u terminates at the same segmentation t_1^j. In both cases, t_1^j is valid since $\mathrm{encode}(\mathrm{decode}(t_1^j)) = \mathrm{encode}(u) = t_1^j$. □

Proof of Lemma 1. Let E be the event that the document begins with x_1^n and, for $\boldsymbol{t} \in \mathrm{cover}(x_1^n)$, let $E_{\boldsymbol{t}}$ be the event that T begins with $\boldsymbol{t}$. ($E \subseteq \bigcup_{\boldsymbol{t}} E_{\boldsymbol{t}}$.) Suppose the document begins with x_1^n, and let k^* be the smallest index with $|\,\mathrm{decode}(T_1^{k^*})| \geq n$, which exists because $\mathrm{decode}(T)$ is the document. Then x_1^n is a prefix of $\mathrm{decode}(T_1^{k^*})$, $\mathrm{decode}(T_1^{k^*-1})$ is a strict prefix of x_1^n by minimality, and $T_1^{k^*}$ is valid by Proposition 1. Hence $T_1^{k^*} \in \mathrm{cover}(x_1^n)$. ($\bigcup_{\boldsymbol{t}} E_{\boldsymbol{t}} \subseteq E$.) If T begins with $\boldsymbol{t} \in \mathrm{cover}(x_1^n)$, the document begins with $\mathrm{decode}(\boldsymbol{t})$, which has x_1^n as a prefix. (Disjointness.) If T begins with two distinct $\boldsymbol{t}, \boldsymbol{t}' \in \mathrm{cover}(x_1^n)$, one of them, say $\boldsymbol{t}$, is a strict prefix of the other, $\boldsymbol{t} = {t'_1}^k$ with $k < |\boldsymbol{t}'|$. Then $\mathrm{decode}({t'_1}^{|\boldsymbol{t}'|-1})$ has $\mathrm{decode}(\boldsymbol{t})$, and hence x_1^n, as a prefix, contradicting the requirement that it be a strict prefix of x_1^n. Therefore $P(x_1^n) = P(E) = \sum_{\boldsymbol{t} \in \mathrm{cover}(x_1^n)} P(E_{\boldsymbol{t}}) = \sum_{\boldsymbol{t} \in \mathrm{cover}(x_1^n)} P(\boldsymbol{t})$. □

Proof of Proposition 3. Let $\boldsymbol{t} = t_1^k \in \mathrm{cover}(x_1^n a)$ and let $j = |\,\mathrm{decode}(t_1^{k-1})| \leq n$. If $j < n$, then $\mathrm{decode}(t_1^{k-1})$ is a strict prefix of x_1^n and x_1^n is a prefix of $\mathrm{decode}(\boldsymbol{t})$, so $\boldsymbol{t} \in \mathrm{cover}(x_1^n)$; moreover $|\,\mathrm{decode}(\boldsymbol{t})| \geq n+1$ and $\mathrm{decode}(\boldsymbol{t})_{n+1} = a$, hence $\boldsymbol{t} \in \mathcal{C}_{n+1}(a)$. If $j = n$, then t_1^{k-1} is valid and decodes to x_1^n, so $t_1^{k-1} = \mathrm{encode}(x_1^n) = \boldsymbol{t}^\star$, and t_k starts with a; hence $\boldsymbol{t} \in \bar{\mathcal{C}}_{n+1}(a)$. Conversely, every element of $\mathcal{C}_{n+1}(a)$ has its second-to-last prefix decoding to a strict prefix of x_1^n, and every element of $\bar{\mathcal{C}}_{n+1}(a)$ has its second-to-last prefix decoding to exactly x_1^n, a strict prefix of $x_1^n a$; both are valid and decode to strings beginning with $x_1^n a$. The two sets are disjoint because their second-to-last prefixes decode to strings of different lengths. Equation 6 then follows from Lemma 1 applied to $x_1^n a$ and x_1^n, using $P(\boldsymbol{t}^\star \oplus \tau) = P(\boldsymbol{t}^\star)P(\tau \mid \boldsymbol{t}^\star)$ and $P(x_1^n) = \sum_b P(x_1^n b)$. □

B ALGORITHMIC DETAILS

Algorithm 2 gives the complete byte-level sampling step corresponding to Figure 4. Starting from the prompt, $\mathrm{cover}(x_1^n)$ is computed by Algorithm 1; each subsequent step reuses the returned cover set.

Algorithm 2 Exact next-byte prediction and sampling from a tokenized LM

Require: byte string x_1^n, $\mathcal{C} = \mathrm{cover}(x_1^n)$ with probabilities $\{P(\boldsymbol{t})\}_{\boldsymbol{t} \in \mathcal{C}}$, tokenized LM P, tokenizer encode

$S(a) \leftarrow 0$ and $\mathcal{C}'(a) \leftarrow \emptyset$ for all $a \in \mathcal{A}$
for each $\boldsymbol{t} \in \mathcal{C}$ with $|\,\mathrm{decode}(\boldsymbol{t})| > n$ **do** ▷ step 2: regroup encodings extending beyond n
  $a \leftarrow \mathrm{decode}(\boldsymbol{t})_{n+1}$; $S(a) \leftarrow S(a) + P(\boldsymbol{t})$; $\mathcal{C}'(a) \leftarrow \mathcal{C}'(a) \cup \{\boldsymbol{t}\}$
end for
$\boldsymbol{t}^\star \leftarrow \mathrm{encode}(x_1^n)$ ▷ the unique element of $\mathcal{C}$ with $|\,\mathrm{decode}(\boldsymbol{t}^\star)| = n$
compute $P(\cdot \mid \boldsymbol{t}^\star)$ with one forward pass ▷ step 3: single LM call
for each $\tau \in \mathcal{V}$ **do**
  if $\mathrm{encode}(\mathrm{decode}(\boldsymbol{t}^\star \oplus \tau)) = \boldsymbol{t}^\star \oplus \tau$ **then** ▷ accept valid continuations only
    $a \leftarrow \tau_1$; $S(a) \leftarrow S(a) + P(\boldsymbol{t}^\star)P(\tau \mid \boldsymbol{t}^\star)$; $\mathcal{C}'(a) \leftarrow \mathcal{C}'(a) \cup \{\boldsymbol{t}^\star \oplus \tau\}$
  end if
end for
$P(a \mid x_1^n) \leftarrow S(a) / \sum_{b \in \mathcal{A}} S(b)$ for all $a \in \mathcal{A}$ ▷ step 4: Equation 6
sample $x_{n+1} \sim P(\cdot \mid x_1^n)$, or take the arg max for greedy decoding ▷ step 5
return x_{n+1} and $\mathrm{cover}(x_1^{n+1}) = \mathcal{C}'(x_{n+1})$ with probabilities $\{P(\boldsymbol{t})\}$

Practical remarks. (i) All probabilities are maintained in log space, and sums are computed with log-sum-exp. (ii) The validity check in Algorithm 2 depends only on the tokenizer. Tokens whose probability is negligible do not affect Equation 6 and can be skipped, which keeps the check inexpensive. (iii) Ideally, $\sum_b S(b) = P(x_1^n)$; for a trained model, the normalization in step 4 discards the mass assigned to invalid encodings. (iv) Special tokens (e.g., FIM sentinels and beginning-of-sequence tokens) are kept fixed as part of the context; the cover search operates only on the text after the last special token. An end-of-sequence token is mapped to a dedicated symbol appended to $\mathcal{A}$. (v) For ensembles, each member runs Algorithm 2 independently up to step 4; the member distributions are averaged, a single byte is selected, and each member keeps its own $\mathcal{C}'(x_{n+1})$. (vi) For byte-level beam search, each hypothesis carries its own cover set; hypotheses are extended by all bytes with non-zero probability and ranked by their cumulative log-probability $\log P(x_1^n)$.

C Relation to token healing and token alignment

Token healing removes the last prompt token, i.e., conditions on $\boldsymbol{p} = \mathrm{encode}(x_1^{n-|t_k|})$ where t_k is the last token of $\mathrm{encode}(x_1^n)$, and restricts the next token to those having the removed bytes as a prefix. In the notation of Proposition 2, this corresponds to retaining a single candidate boundary j and discarding all others; furthermore, the retained boundary is chosen by the tokenizer rather than weighted by the model probability $P(\mathrm{encode}(x_1^j))$, and invalid continuations are not excluded. Token alignment extends this idea by backing up several tokens and constraining the generated tokens to match the removed characters, which again considers a subset of the cover encodings without weighting them according to Lemma 1. Our method sums over every candidate boundary $j \in \{n-L, \ldots, n-1\}$ with exact weights, as well as over all valid continuations of $\mathrm{encode}(x_1^n)$ during generation (Proposition 3), and therefore recovers the correct conditional distribution under the ideal model. Empirically, this difference translates into the gains over both heuristics in SPM mode reported in Figure 6.